ADP 실기 대비 핵심 요약: [1] 가설 수립 및 평균 차이 검정 (모수/비모수)

1. 개요 및 출제 의도

ADP 실기 시험의 통계 분석 영역(배점 50 점)에서 가장 기본적이면서 빈출되는 유형입니다. 단순히 검정 함수를 호출하는 것에 그치지 않고, 아래의 4 단계 분석 절차를 명확하게 준수해야 감점을 피할 수 있습니다.

1. **가설 수립**: 문제 배경에 맞는 귀무가설(H_0) 및 대립가설(H_1) 명시 (양측/단측 검정 구분)
2. **사전 가정 검정**:
   - 정규성 검정 (Shapiro-Wilk Test 등)
   - 등분산성 검정 (Levene Test, Bartlett Test 등)
3. **통계 검정 기법 선택 및 수행**:
   - 정규성 만족 + 등분산 만족 → 독립표본 t-검정 (`equal_var=True`)
   - 정규성 만족 + 등분산 불만족 → 웰치스 t-검정 (`equal_var=False`)
   - 정규성 불만족 → 만-위트니 U 검정 (Mann-Whitney U Test)
4. **검정통계량 제시 및 비즈니스 결론 서술**: 유의수준(α=0.05)과 p-value 비교를 통한 가설 채택/기각 서술

---

2. 실전 기출 유형 문제

[문제]

한 제약회사에서 새로 개발한 혈압 강하제(신약)의 효과를 검증하고자 합니다. 환자 100 명을 무작위로 선별하여 50 명에게는 기존 표준 치료제(Standard)를, 나머지 50 명에게는 신약(New)을 투여한 후 혈압 강하량(수축기 혈압 감소 폭, mmHg)을 측정하였습니다.

1. 두 집단 간 혈압 강하량의 평균 차이가 존재하는지 확인하기 위한 **귀무가설(H_0)과 대립가설(H_1)을 수립**하시오.
2. 두 집단의 데이터에 대해 **정규성 검정 및 등분산성 검정을 수행**하고 결과를 해석하시오.

3. 검정 결과에 따라 적절한 통계 검정 기법을 선택하여 분석을 수행하고, **검정통계량 및 p-value 를 제시**하시오.
4. 유의수준 0.05 하에서 통계적 의사결정을 내리고, **신약의 효과에 대한 최종 결론을 서술**하시오.

---

3. 완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from scipy import stats

# 1. 시뮬레이션 데이터셋 생성 (실제 Seaborn / Scipy 데이터 구조 반영)
np.random.seed(42)
n = 50

# 표준 치료제 그룹 (정규분포를 따르는 데이터)
standard_group = np.random.normal(loc=12.5, scale=4.0, size=n)

# 신약 그룹 (혈압 강하 효과가 더 높은 데이터)
new_group = np.random.normal(loc=15.2, scale=4.5, size=n)

df = pd.DataFrame({
    'treatment': ['Standard'] * n + ['New'] * n,
    'bp_reduction': np.concatenate([standard_group, new_group])
})

std_data = df[df['treatment'] == 'Standard']['bp_reduction']
new_data = df[df['treatment'] == 'New']['bp_reduction']

# ========================================
# [소문항 1] 가설 수립
# ========================================
print("=== [1] 가설 수립 ===")
print("귀무가설 (H0): 기존 표준 치료제와 신약 간의 평균 혈압 강하량에는 차이가 없다. (μ_standard = μ_new)")
print("대립가설 (H1): 기존 표준 치료제와 신약 간의 평균 혈압 강하량에는 차이가 있다. (μ_standard != μ_new)\n")

# ========================================
# [소문항 2] 정규성 및 등분산성 검정
# ========================================
print("=== [2] 사전 가정 검정 ===")

# (1) 정규성 검정 (Shapiro-Wilk Test)
stat_std, p_std = stats.shapiro(std_data)
```

```
stat_new, p_new = stats.shapiro(new_data)

print(f"[정규성] Standard 그룹 - 통계량: {stat_std:.4f}, p-value: {p_std:.4f}")
print(f"[정규성] New 그룹 - 통계량: {stat_new:.4f}, p-value: {p_new:.4f}")

# (2) 등분산성 검정 (Levene Test)
stat_levene, p_levene = stats.levene(std_data, new_data)
print(f"[등분산성] Levene 검정 - 통계량: {stat_levene:.4f}, p-value:
{p_levene:.4f}\n")

# ========================================
# [소문항 3 & 4] 검정 기법 선택, 수행 및 최종 해석
# ========================================
print("=== [3 & 4] 가설검정 수행 및 최종 결론 ===")

alpha = 0.05

# 정규성 만족 여부 판단
if p_std > alpha and p_new > alpha:
    print("-> 두 집단 모두 정규성을 만족하므로 '독립표본 t-검정(Two-sample t-
test)'을 수행합니다.")

    # 등분산 만족 여부에 따른 분기
    if p_levene > alpha:
        t_stat, p_val = stats.ttest_ind(new_data, std_data, equal_var=True)
        print("-> 등분산성을 만족하므로 Student's t-test 를 적용합니다.")
    else:
        t_stat, p_val = stats.ttest_ind(new_data, std_data, equal_var=False)
        print("-> 등분산성을 만족하지 않으므로 Welch's t-test 를 적용합니다.")

    print(f"검정통계량 (t-statistic): {t_stat:.4f}")
    print(f"유의확률 (p-value): {p_val:.4e}")

else:
    print("-> 정규성을 만족하지 않으므로 비모수 검정인 '만-위트니 U 검정(Mann-
Whitney U Test)'을 수행합니다.")
    u_stat, p_val = stats.mannwhitneyu(new_data, std_data, alternative='two-
sided')
    print(f"검정통계량 (U-statistic): {u_stat:.4f}")
    print(f"유의확률 (p-value): {p_val:.4e}")

# 최종 의사결정 서술
if p_val < alpha:
    print(f"\n[최종 결론]: 유의확률(p-value = {p_val:.4f})이 유의수준 0.05 보다
```

```
작으므로 귀무가설(H0)을 기각하고 대립가설(H1)을 채택합니다.")
    print("따라서 기존 표준 치료제와 신약 투여군 간의 혈압 강하량에는 통계적으로 유의미한 차이가 존재합니다.")
else:
    print(f"\n[최종 결론]: 유의확률(p-value = {p_val:.4f})이 유의수준 0.05 보다 크므로 귀무가설(H0)을 기각할 수 없습니다.")
    print("따라서 두 집단 간의 혈압 강하량에 유의미한 차이가 있다고 볼 수 없습니다.")
```

---

4. 실기 시험 채점 포인트 및 주의사항

- **가설 수립 서술**:
- 문맥에 따라 단측 검정인지(예: '신약의 효과가 더 큰지 검정하시오' → μ_new > μ_std), 양측 검정인지(예: '차이가 있는지 검정하시오' → μ_new ≠ μ_std) 명확히 구별해야 합니다.
- **파이프라인 분기 누락 방지**:
- 실기 채점관은 t-test 결과값만 보지 않고, 정규성 검정과 등분산 검정을 제대로 수행하여 분기를 탔는지를 엄격히 채점합니다.
- **비모수 검정 대체**:
- 단일/대응표본: `scipy.stats.wilcoxon`
- 독립 2 표본: `scipy.stats.mannwhitneyu`

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 2. [1] 통계 분석의 핵심인 '분산분석 (ANOVA) 및 사후검정(Post-Hoc)과 비모수 분기(Kruskal-Wallis)'입니다.

3 개 이상의 독립 집단 간 평균 차이를 검정할 때 사용하며, 일원배치 분산분석 → 유의성 확인 → 사후검정(Tukey/Scheffe 등)으로 이어지는 흐름과 정규성이 위배될 때 비모수 검정(Kruskal-Wallis → Dunn's test)으로 전환하는 파이프라인이 빈출됩니다.

실전 기출 유형 문제

[문제] 한 전자상거래 플랫폼에서 3 가지 웹사이트 UI 디자인(A, B, C)에 따른 고객들의 체류 시간(초) 차이를 분석하고자 합니다. 각 디자인 안별로 40 명씩 총 120 명의 고객을 무작위 배정하여 체류 시간을 측정했습니다. 1. 세 UI 디자인 간 고객 평균 체류 시간에 차이가 있는지 확인하기 위한 **귀무가설(H_0)과 대립가설(H_1)을 수립**하시오.

2. 각 그룹의 **정규성 및 세 그룹 간 등분산성 검정을 수행**하고 결과를 해석하시오.
3. 가정 만족 여부에 따라 적절한 검정을 수행하여 **통계량 및 p-value 를 제시하고 결론을 서술**하시오.
4. 귀무가설이 기각될 경우, **어느 그룹 간에 유의미한 차이가 있는지 사후검정(Post-Hoc Test)을 수행**하고 결과를 해석하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# 1. 온라인/시뮬레이션 데이터셋 생성 (실제 Seaborn / Scipy 데이터 구조 반영)
np.random.seed(42)
n_per_group = 40

group_A = np.random.normal(loc=120, scale=15, size=n_per_group)
group_B = np.random.normal(loc=122, scale=15, size=n_per_group)
group_C = np.random.normal(loc=135, scale=15, size=n_per_group)  # 체류 시간이 더 긴 그룹

df = pd.DataFrame({
    'ui_version': ['A'] * n_per_group + ['B'] * n_per_group + ['C'] * n_per_group,
    'dwell_time': np.concatenate([group_A, group_B, group_C])
})

data_A = df[df['ui_version'] == 'A']['dwell_time']
data_B = df[df['ui_version'] == 'B']['dwell_time']
data_C = df[df['ui_version'] == 'C']['dwell_time']

# ========================================
# [소문항 1] 가설 수립
# ========================================
```

```
print("=== [1] 가설 수립 ===")
print("귀무가설 (H0): 3 가지 UI 디자인에 따른 고객의 평균 체류 시간은 모두 같다. (μ_A = μ_B = μ_C)")
print("대립가설 (H1): 3 가지 UI 디자인 중 적어도 한 UI 의 평균 체류 시간은 다르다.\n")

# ========================================
# [소문항 2] 정규성 및 등분산성 검정
# ========================================
print("=== [2] 사전 가정 검정 ===")

# (1) 정규성 검정 (Shapiro-Wilk)
p_norm_A = stats.shapiro(data_A).pvalue
p_norm_B = stats.shapiro(data_B).pvalue
p_norm_C = stats.shapiro(data_C).pvalue

print(f"[정규성] UI A p-value: {p_norm_A:.4f}, UI B p-value: {p_norm_B:.4f}, UI C p-value: {p_norm_C:.4f}")

# (2) 등분산성 검정 (Levene Test)
stat_levene, p_levene = stats.levene(data_A, data_B, data_C)
print(f"[등분산성] Levene 검정 - 통계량: {stat_levene:.4f}, p-value: {p_levene:.4f}\n")

# ========================================
# [소문항 3] 검정 수행 및 결론
# ========================================
print("=== [3] 가설검정 수행 ===")
alpha = 0.05

is_normal = (p_norm_A > alpha) and (p_norm_B > alpha) and (p_norm_C > alpha)
is_equal_var = (p_levene > alpha)

if is_normal and is_equal_var:
    print("-> 정규성과 등분산성을 모두 만족하므로 '일원배치 분산분석(One-way ANOVA)'을 수행합니다.")
    f_stat, p_val = stats.f_oneway(data_A, data_B, data_C)
    print(f"F-통계량: {f_stat:.4f}, p-value: {p_val:.4e}")

    if p_val < alpha:
        print(f"[결론]: p-value({p_val:.4e}) < 0.05 이므로 귀무가설을 기각합니다.")
        print("즉, UI 디자인 간 고객 평균 체류 시간에 유의미한 차이가 존재합니다.\n")
```

```
        # ========================================
        # [소문항 4] 사후검정 (Tukey HSD)
        # ========================================
        print("=== [4] 사후검정 (Tukey HSD) ===")
        tukey_result = pairwise_tukeyhsd(endog=df['dwell_time'],
groups=df['ui_version'], alpha=0.05)
        print(tukey_result)
        print("\n[사후검정 해석]: reject=True 인 그룹 간에 유의미한 평균 차이가
있습니다.")
    else:
        print(f"[결론]: p-value({p_val:.4f}) >= 0.05 이므로 귀무가설을 기각할
수 없습니다.")
        print("UI 디자인 간 체류 시간 차이가 유의미하지 않으므로 사후검정을
수행하지 않습니다.")

else:
    print("-> 정규성 또는 등분산성이 위배되었으므로 비모수 검정인 '크루스칼-왈리스
검정(Kruskal-Wallis Test)'을 수행합니다.")
    kw_stat, p_val = stats.kruskal(data_A, data_B, data_C)
    print(f"H-통계량: {kw_stat:.4f}, p-value: {p_val:.4e}")

    if p_val < alpha:
        print(f"[결론]: p-value({p_val:.4e}) < 0.05 이므로 귀무가설을
기각합니다.")
        print("비모수 사후검정(Dunn's test 등)을 통해 상세 그룹 차이를
확인합니다.")
```

ADP 실기 채점 포인트 요약

- **대립가설 작성 오류 방지**: ANOVA 대립가설 작성 시 "모든 집단의 평균이 다르다"가 아니라 "적어도 한 집단의 평균은 다르다"라고 명시해야 감점되지 않습니다.
- **사후검정 실행 조건**: 분산분석 본 검정에서 p-value 가 0.05 미만으로 귀무가설이 기각되었을 때만 사후검정을 진행해야 합니다.
- **사후검정 결과표 해석**: `statsmodels` 의 `pairwise_tukeyhsd` 출력 결과에서 `reject` 컬럼이 `True` 인 비교군(예: A-C, B-C)을 짚어서 명확한 문장으로 적어주어야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 3. [1] 통계 분석의 '범주형 자료분석 (교차분석: 카이제곱 독립성/동질성/적합도 검정 및 피셔의 정확검정)'입니다.

실제 실기 시험에서는 두 범주형 변수 간의 연관성 유무를 검정하는 **카이제곱 독립성 검정**이 가장 빈출되며, 기대빈도가 5 미만인 셀이 전체의 20%를 초과할 경우 피셔의 정확검정(Fisher's Exact Test)으로 전환하는 흐름이 핵심 채점 포인트입니다.

실전 기출 유형 문제

[문제] 통신사에서 고객의 연령대(20 대, 30 대, 40 대 이상)에 따라 선호하는 요금제 유형(알뜰 요금제, 표준 요금제, 프리미엄 요금제) 간에 연관성이 존재하는지 분석하고자 합니다. 고객 300 명을 표본 조사하여 교차표를 생성했습니다. 1. 연령대와 선호 요금제 유형 간의 독립성을 검정하기 위한 **귀무가설(H_0)과 대립가설(H_1)을 수립**하시오.

2. 원시 데이터를 기반으로 **교차분할표(Contingency Table)를 생성**하고, 각 셀의 **기대빈도(Expected Frequency)를 확인**하시오.
3. 카이제곱 독립성 검정을 수행하여 **검정통계량, 자유도, p-value 를 제시**하시오.
4. 유의수준 0.05 하에서 결과를 해석하고, **연령대와 요금제 선호도 간의 관계에 대한 최종 결론을 서술**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from scipy import stats

# 1. 온라인/시뮬레이션 데이터셋 생성
np.random.seed(42)
n_samples = 300

age_groups = ['20s', '30s', '40s_above']
plans = ['Budget', 'Standard', 'Premium']

# 연령대별 요금제 선택 확률 다르게 설정 (20 대는 Budget, 40 대는 Premium 선호 경향)
age_sample = np.random.choice(age_groups, size=n_samples, p=[0.35, 0.35,
0.30])
plan_sample = []
```

```
for age in age_sample:
    if age == '20s':
        plan = np.random.choice(plans, p=[0.60, 0.30, 0.10])
    elif age == '30s':
        plan = np.random.choice(plans, p=[0.30, 0.50, 0.20])
    else:  # 40s_above
        plan = np.random.choice(plans, p=[0.15, 0.35, 0.50])
    plan_sample.append(plan)

df = pd.DataFrame({'age_group': age_sample, 'plan_type': plan_sample})

# ========================================
# [소문항 1] 가설 수립
# ========================================
print("=== [1] 가설 수립 ===")
print("귀무가설 (H0): 고객의 연령대와 선호 요금제 유형은 서로 독립이다. (연관성이 없다.)")
print("대립가설 (H1): 고객의 연령대와 선호 요금제 유형은 서로 독립이 아니다. (연관성이 있다.)\n")

# ========================================
# [소문항 2] 교차분할표 생성 및 기대빈도 확인
# ========================================
print("=== [2] 교차분할표 및 기대빈도 ===")
contingency_table = pd.crosstab(df['age_group'], df['plan_type'])
print("[관측 빈도표 (Observed Table)]")
print(contingency_table)

# 카이제곱 독립성 검정 수행
chi2_stat, p_val, dof, expected = stats.chi2_contingency(contingency_table)

expected_df = pd.DataFrame(expected, index=contingency_table.index, columns=contingency_table.columns)
print("\n[기대 빈도표 (Expected Table)]")
print(expected_df.round(2))

# 기대빈도 5 미만 셀 비율 확인 (가정 검토)
cells_less_than_5 = (expected < 5).sum()
total_cells = expected.size
ratio = (cells_less_than_5 / total_cells) * 100
print(f"\n기대빈도 5 미만 셀 비율: {ratio:.1f}% ({cells_less_than_5}/{total_cells}개)")

# ========================================
# [소문항 3 & 4] 검정 결과 및 최종 결론
# ========================================
```

```
print("\n=== [3 & 4] 검정 결과 및 최종 해석 ===")
alpha = 0.05

print(f"카이제곱 검정통계량 (Chi2): {chi2_stat:.4f}")
print(f"자유도 (df): {dof}")
print(f"유의확률 (p-value): {p_val:.4e}")

if ratio > 20:
    print("\n* 주의: 기대빈도 5 미만인 셀이 20%를 초과하므로 2x2 테이블인 경우 Fisher's Exact Test 적용을 권장합니다.")

if p_val < alpha:
    print(f"\n[최종 결론]: 유의확률(p-value = {p_val:.4e})이 유의수준 0.05 보다 작으므로 귀무가설(H0)을 기각하고 대립가설(H1)을 채택합니다.")
    print("따라서 고객의 연령대와 선호 요금제 유형 간에는 통계적으로 유의미한 연관성(종속 관계)이 존재합니다.")
else:
    print(f"\n[최종 결론]: 유의확률(p-value = {p_val:.4f})이 유의수준 0.05 보다 크므로 귀무가설(H0)을 기각할 수 없습니다.")
    print("따라서 고객의 연령대와 선호 요금제 유형 간에 유의미한 연관성이 있다고 볼 수 없습니다.")
```

ADP 실기 채점 포인트 요약

- **교차표 형태 입력**: `stats.chi2_contingency()`에는 원시 데이터프레임이 아니라 반드시 `pd.crosstab()`으로 생성한 2 차원 빈도 테이블을 전달해야 합니다.
- **기대빈도 가정 명시**: 카이제곱 검정의 전제 조건인 "기대빈도가 5 미만인 셀이 20%를 넘지 않아야 한다"는 점을 기대빈도 배열(expected)을 통해 확인하는 과정을 서술하면 가산점을 받습니다.
- **자유도(df) 계산 확인**: (행의 수 – 1) × (열의 수 – 1) 공식에 맞게 산출되었는지 확인하고 결과 서술 시 함께 기재해야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 4. [1] 통계 분석의 핵심인 '다중선형회귀분석 및 모델 가정 검정(선형성, 등분산성, 정규성, 독립성, 다중공선성)'입니다.

실기 시험에서 가장 배점이 높고 요구하는 하위 문항이 많은 유형으로, **회귀식 도출 → 통계적 유의성(F-통계량, t-검정) → 다중공선성(VIF) 진단 → 잔차 4 대 가정 검정 → 최종 해석** 전 과정을 단계별로 작성해야 합니다.

실전 기출 유형 문제

[문제] 한 중고차 거래 플랫폼에서 차량 가격(price, 만원)을 예측하기 위해 주행거리(mileage, 천 km), 연식(age, 년), 엔진 크기(engine_size, L) 데이터를 수집했습니다. 1. 세 독립변수를 활용하여 차량 가격을 종속변수로 하는 **다중선형회귀모형을 적합하고 회귀식을 제시**하시오.

2. 모형의 **결정계수(R^2), 수정 결정계수(Adj-R^2), 회귀모형 전체의 유의성(F-통계량 및 p-value)**을 제시하고 해석하시오.
3. 각 독립변수의 **회귀계수 유의성(t-검정 p-value)을 확인하고, VIF(분산팽창지수)를 산출하여 다중공선성을 진단**하시오.
4. 잔차에 대해 **정규성(Shapiro-Wilk), 등분산성(Breusch-Pagan), 독립성(Durbin-Watson) 검정을 수행**하고 가정이 충족되는지 서술하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson
from scipy import stats

# 1. 온라인/시뮬레이션 데이터셋 생성
np.random.seed(42)
n = 150

mileage = np.random.uniform(10, 200, size=n)       # 주행거리 (천km)
age = np.random.uniform(1, 15, size=n)             # 연식 (년)
engine_size = np.random.uniform(1.0, 3.5, size=n) # 엔진 크기 (L)
```

```python
# 회귀식: Price = 3000 - 8*mileage - 70*age + 400*engine_size + noise
noise = np.random.normal(0, 150, size=n)
price = 3000 - 8 * mileage - 70 * age + 400 * engine_size + noise

df = pd.DataFrame({
    'price': price,
    'mileage': mileage,
    'age': age,
    'engine_size': engine_size
})

# ========================================
# [소문항 1 & 2] 회귀모형 적합 및 모형 유의성
# ========================================
print("=== [1 & 2] 다중선형회귀모형 적합 및 유의성 ===")
model = ols('price ~ mileage + age + engine_size', data=df).fit()

print(model.summary())

intercept = model.params['Intercept']
b_mileage = model.params['mileage']
b_age = model.params['age']
b_engine = model.params['engine_size']

print(f"\n[도출된 회귀식]: price = {intercept:.2f} + ({b_mileage:.2f})*mileage
+ ({b_age:.2f})*age + ({b_engine:.2f})*engine_size")
print(f"결정계수 (R-squared): {model.rsquared:.4f}")
print(f"수정 결정계수 (Adj. R-squared): {model.rsquared_adj:.4f}")
print(f"F-통계량: {model.fvalue:.4f}, p-value: {model.f_pvalue:.4e}")

if model.f_pvalue < 0.05:
    print("-> F-통계량의 p-value 가 0.05 미만이므로 본 회귀모형은 통계적으로
유의합니다.")

# ========================================
# [소문항 3] 다중공선성 (VIF) 산출
# ========================================
print("\n=== [3] 다중공선성 진단 (VIF) ===")
X = df[['mileage', 'age', 'engine_size']]
X_with_const = sm.add_constant(X)

vif_df = pd.DataFrame()
vif_df['Variable'] = X_with_const.columns
vif_df['VIF'] = [variance_inflation_factor(X_with_const.values, i) for i in
range(X_with_const.shape[1])]

print(vif_df)
print("-> VIF 기준(보통 10 미만)을 확인했을 때, 모든 독립변수의 VIF 가 10
```

```
미만이므로 다중공선성 문제는 없습니다.")

# ========================================
# [소문항 4] 잔차 가정 검정 (정규성, 등분산성, 독립성)
# ========================================
print("\n=== [4] 잔차 4대 가정 검정 ===")
residuals = model.resid

# 1. 정규성 검정 (Shapiro-Wilk)
shapiro_stat, shapiro_p = stats.shapiro(residuals)
print(f"[정규성] Shapiro-Wilk p-value: {shapiro_p:.4f} -> {'만족 (p >= 0.05)'
if shapiro_p >= 0.05 else '불만족 (p < 0.05)'}")

# 2. 등분산성 검정 (Breusch-Pagan)
bp_test = het_breuschpagan(residuals, model.model.exog)
bp_pvalue = bp_test[1]
print(f"[등분산성] Breusch-Pagan p-value: {bp_pvalue:.4f} -> {'만족 (p >=
0.05)' if bp_pvalue >= 0.05 else '불만족 (p < 0.05)'}")

# 3. 독립성 검정 (Durbin-Watson)
dw_stat = durbin_watson(residuals)
print(f"[독립성] Durbin-Watson 통계량: {dw_stat:.4f} -> 2에 근접하므로
자기상관(독립성 위배) 없음")
```

ADP 실기 채점 포인트 요약

- **statsmodels vs sklearn**: 통계 검정, p-value, F-통계량, 잔차 진단은 Scikit-Learn 이 아닌 `statsmodels.formula.api.ols` 를 사용하는 것이 훨씬 수월하고 결과표를 그대로 답안에 활용할 수 있습니다.
- **회귀식 서술**: 절편(Intercept)과 각 변수별 기울기 부호(+, -)를 명확히 포함하여 완전한 수식 형태로 기재해야 감점되지 않습니다.
- **잔차 진단 패키지**: het_breuschpagan(등분산성), durbin_watson(독립성), shapiro(정규성)의 사용법과 p-value 판정 기준을 암기해 두어야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 5. [1] 통계 분석의 고배점 빈출 영역인 '로지스틱 회귀분석 (Logistic Regression) 및 오즈비(Odds Ratio) 해석'입니다.

실기 시험에서는 분류 모델로서 머신러닝 관점뿐 아니라, 통계적 관점의 로지스틱 회귀분석(회귀계수 유의성, 오즈비 산출, 모형 적합도)을 묻는 문제가 자주 출제됩니다.

실전 기출 유형 문제

[문제] 은행에서 고객의 대출 채무 불이행(default, 0: 정상, 1: 연체) 여부를 예측하고 주요 위험 요인을 분석하고자 합니다. 고객 200 명의 연소득(income, 천만원), 신용점수(credit_score), 기존 부채 비율(debt_ratio, %) 데이터를 수집했습니다. 1. 세 독립변수를 사용하여 default 를 종속변수로 하는 **로지스틱 회귀모형을 적합하고 통계 요약표를 제시**하시오.

2. 모형의 **Log-Likelihood, Pseudo R^2 (McFadden), AIC 값을 제시하고 모형의 유의성을 서술**하시오.
3. 각 독립변수의 **회귀계수에 대해 지수 변환(e^β)을 적용하여 오즈비(Odds Ratio)를 산출하고, 신용점수와 부채 비율이 연체 확률에 미치는 영향을 비즈니스 관점에서 해석**하시오.
4. 신용점수가 650 점, 부채 비율이 30%, 연소득이 4 천만원인 고객의 **연체 예측 확률을 계산**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import logit

# 1. 시뮬레이션 데이터셋 생성
np.random.seed(42)
n = 200

income = np.random.uniform(2.0, 10.0, size=n)          # 연소득 (천만원)
credit_score = np.random.uniform(500, 850, size=n)  # 신용점수
debt_ratio = np.random.uniform(10, 80, size=n)       # 부채 비율 (%)

# 로짓(Log-odds) 계산: 신용점수/소득 높을수록 연체 감소, 부채비율 높을수록 연체 증가
```

```
z = 3.0 - 0.3 * income - 0.01 * credit_score + 0.08 * debt_ratio +
np.random.normal(0, 0.5, size=n)
prob = 1 / (1 + np.exp(-z))
default = (prob > 0.5).astype(int)

df = pd.DataFrame({
    'default': default,
    'income': income,
    'credit_score': credit_score,
    'debt_ratio': debt_ratio
})

# ========================================
# [소문항 1 & 2] 로지스틱 회귀 적합 및 모델 지표
# ========================================
print("=== [1 & 2] 로지스틱 회귀 적합 및 모델 지표 ===")
model = logit('default ~ income + credit_score + debt_ratio', data=df).fit()

print(model.summary())

print(f"\nLog-Likelihood: {model.llf:.4f}")
print(f"Pseudo R-squared (McFadden): {model.prsquared:.4f}")
print(f"AIC: {model.aic:.4f}")
print(f"LLR p-value: {model.llr_pvalue:.4e}")

if model.llr_pvalue < 0.05:
    print("-> LLR p-value 가 0.05 미만이므로 본 로지스틱 회귀모형은 통계적으로
유의합니다.")

# ========================================
# [소문항 3] 회귀계수 지수 변환 및 오즈비(Odds Ratio) 해석
# ========================================
print("\n=== [3] 오즈비 (Odds Ratio) 산출 및 해석 ===")
params = model.params
odds_ratios = np.exp(params)
p_values = model.pvalues

odds_df = pd.DataFrame({
    'Coefficient': params,
    'Odds_Ratio (exp)': odds_ratios,
    'p-value': p_values
})
print(odds_df.round(4))

print("\n[오즈비 해석]:")
for col in ['income', 'credit_score', 'debt_ratio']:
    or_val = odds_ratios[col]
    if or_val > 1:
        pct_change = (or_val - 1) * 100
        print(f"- {col}: 1 단위 증가할 때 연체 오즈(Odds)가 약 {pct_change:.2f}%
```

```
증가합니다. (OR = {or_val:.4f})")
    else:
        pct_change = (1 - or_val) * 100
        print(f"- {col}: 1 단위 증가할 때 연체 오즈(Odds)가 약 {pct_change:.2f}%
감소합니다. (OR = {or_val:.4f})")

# ========================================
# [소문항 4] 특정 조건 고객의 예측 확률 계산
# ========================================
print("\n=== [4] 특정 조건 고객의 연체 확률 계산 ===")
new_customer = pd.DataFrame({
    'income': [4.0],
    'credit_score': [650.0],
    'debt_ratio': [30.0]
})

pred_prob = model.predict(new_customer)[0]
print(f"새로운 고객의 예측 연체 확률: {pred_prob:.4f} ({pred_prob*100:.2f}%)")
```

ADP 실기 채점 포인트 요약

- **오즈비(Odds Ratio) 계산**: 로지스틱 회귀계수(β)는 로그-오즈(Log-odds) 변화량이므로, 반드시 `np.exp(model.params)`로 지수 변환하여 오즈비를 구하고 서술해야 합니다.
- **오즈비 증감 퍼센트 서술**:
- Odds Ratio > 1: (OR − 1) × 100% 증가
- Odds Ratio < 1: (1 − OR) × 100% 감소
- **모형 적합도**: `model.llr_pvalue`(우도비 검정 유의확률)와 `model.prsquared`(의사 결정계수)를 확인하여 통계적 유의성을 기재해야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 6. [1] 통계 분석의 단골 고난도 문항인 '시계열 분석 (Time Series Analysis: 정상성 검정, 차분, ARIMA/SARIMA 모형 식별 및 잔차 진단)'입니다.

실기 시험에서는 **(1) 정상성 검정(ADF Test) → (2) 비정상 시 차분 수행 → (3) ACF/PACF 플롯을 통한 ARIMA(p, d, q) 차수 식별 → (4) 모델 적합 및 잔차 백색잡음 검정(Ljung-Box Test) → (5) 미래 시점 예측**의 전 과정을 순서대로 코딩하고 서술해야 합니다.

실전 기출 유형 문제

[문제] 한 전력 공급 기업의 월별 전력 소비량 시계열 데이터를 분석하여 향후 전력 수요를 예측하고자 합니다. 120 개월 분량의 월별 전력 사용량 데이터가 주어졌습니다. 1. 원시계열 데이터에 대해 **ADF(Augmented Dickey-Fuller) 검정을 수행하여 정상성(Stationarity) 만족 여부를 판정**하시오.

2. 비정상 시계열일 경우, **적절한 차분(Differencing)을 수행하고 차분 후의 데이터에 대해 다시 정상성 검정을 수행**하시오.
3. 차분된 데이터의 **ACF 및 PACF 플롯을 분석하여 적합한 ARIMA(p, d, q) 모형의 후보 차수를 결정**하시오.
4. 결정된 차수로 **ARIMA 모형을 적합하고, 잔차(Residuals)에 대해 Ljung-Box 검정을 수행하여 모형의 적합성을 진단**하시오.
5. 적합된 모형을 기반으로 **향후 12 개월간의 전력 소비량을 예측**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox

# 1. 온라인/시뮬레이션 시계열 데이터셋 생성 (추세 + AR(1) 노이즈)
np.random.seed(42)
n = 120

time_idx = pd.date_range(start='2016-01-01', periods=n, freq='MS')
# 비정상 시계열: 선형 추세(Trend) + AR(1) 구조
trend = np.linspace(50, 120, n)
ar_noise = np.zeros(n)
for t in range(1, n):
    ar_noise[t] = 0.6 * ar_noise[t-1] + np.random.normal(0, 3)
```

```
ts_data = pd.Series(trend + ar_noise, index=time_idx,
name='electricity_demand')

# ========================================
# [소문항 1] 원계열 정상성 검정 (ADF Test)
# ========================================
print("=== [1] 원계열 정상성 검정 (ADF Test) ===")
def run_adf(series, name="Series"):
    result = adfuller(series.dropna())
    print(f"[{name}] ADF 통계량: {result[0]:.4f}, p-value: {result[1]:.4e}")
    if result[1] < 0.05:
        print(f"-> p-value < 0.05 이므로 귀무가설(단위근 존재)을 기각합니다.
({name}은 정상 시계열)")
        return True
    else:
        print(f"-> p-value >= 0.05 이므로 귀무가설을 기각할 수 없습니다.
({name}은 비정상 시계열)")
        return False

is_stationary = run_adf(ts_data, "원계열")

# ========================================
# [소문항 2] 차분(Differencing) 및 정상성 재검정
# ========================================
print("\n=== [2] 1 차 차분 수행 및 정상성 검정 ===")
ts_diff = ts_data.diff().dropna()
is_diff_stationary = run_adf(ts_diff, "1 차 차분 계열")

d_order = 1 if is_diff_stationary else 2

# ========================================
# [소문항 3 & 4] ARIMA 모형 적합 및 잔차 진단
# ========================================
print("\n=== [3 & 4] ARIMA(1, 1, 1) 모형 적합 및 잔차 진단 ===")
# 실기 환경에서는 ACF/PACF 형태를 보고 p, q 를 결정 (예: p=1, d=1, q=1)
p_order, q_order = 1, 1

model = ARIMA(ts_data, order=(p_order, d_order, q_order))
model_fit = model.fit()

print(model_fit.summary())

# 잔차 진단: Ljung-Box 검정 (잔차의 자기상관 여부 / 백색잡음 검정)
residuals = model_fit.resid
lb_test = acorr_ljungbox(residuals, lags=[10], return_df=True)
lb_pvalue = lb_test['lb_pvalue'].values[0]
```

```
print(f"\n[Ljung-Box 잔차 진단] lags=10 p-value: {lb_pvalue:.4f}")
if lb_pvalue > 0.05:
    print("-> p-value > 0.05 이므로 귀무가설(잔차에 자기상관이 없다/백색잡음이다)을 채택합니다.")
    print("따라서 ARIMA 모형이 시계열 패턴을 충분히 설명하고 있으며 모형이 적합합니다.")
else:
    print("-> p-value <= 0.05 이므로 잔차에 자기상관이 남아있어 모형 차수 조정이 필요합니다.")

# ========================================
# [소문항 5] 미래 시점 예측
# ========================================
print("\n=== [5] 향후 12 개월 전력 수요 예측 ===")
forecast_steps = 12
forecast_res = model_fit.get_forecast(steps=forecast_steps)
forecast_mean = forecast_res.predicted_mean
conf_int = forecast_res.conf_int(alpha=0.05)

forecast_df = pd.DataFrame({
    'Forecast': forecast_mean,
    'Lower_95%': conf_int.iloc[:, 0],
    'Upper_95%': conf_int.iloc[:, 1]
})
print(forecast_df.head(6))
```

ADP 실기 채점 포인트 요약

- **ADF 가설 방향 암기**:
- 귀무가설(H_0): 단위근이 존재한다 (비정상 시계열이다)
- 대립가설(H_1): 단위근이 존재하지 않는다 (정상 시계열이다)
- 따라서 **p-value < 0.05 이어야 정상성 만족**입니다.
- **Ljung-Box 가설 방향 암기 (ADF 와 반대)**:
- 귀무가설(H_0): 잔차들이 서로 독립이다 (백색잡음 만족, 모형 적합)
- 대립가설(H_1): 잔차에 자기상관이 존재한다 (모형 부적합)
- 따라서 **p-value > 0.05 이어야 모형 적합**입니다.

- **`statsmodels.tsa.arima.model.ARIMA` 활용**: 최신 버전에서는 과거 `ARIMA` 대신 일원화된 `statsmodels.tsa.arima.model.ARIMA` 클래스를 사용해야 경고 없이 정확히 동작합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 7. [2] 데이터 전처리 및 특성 공학의 핵심인 '데이터 누수(Data Leakage) 방지 파이프라인(ColumnTransformer + Pipeline) 및 클래스 불균형 처리(SMOTE)'입니다.

실기 시험에서는 머신러닝 문제(50 점)의 첫 단계로 전처리가 주어지며, **Train 데이터의 통계량(평균, 스케일링 기준 등)이 Test 데이터로 누수되지 않도록 Pipeline 구조로 묶는 것**과 **불균형 데이터에 대해 imbalanced-learn 의 SMOTE 를 올바르게 적용하는 것**이 감점을 피하는 핵심 기준입니다.

실전 기출 유형 문제

[문제] 고객의 서비스 이탈(churn, 0: 유지, 1: 이탈) 여부를 예측하기 위한 원본 데이터가 주어졌습니다. 이탈 비율이 전체의 약 15%로 클래스 불균형이 존재하며, 수치형 변수와 범주형 변수가 혼합되어 있고 결측치 및 이상치가 포함되어 있습니다. 1. 데이터를 학습용(`X_train`, `y_train`)과 평가용(`X_test`, `y_test`)으로 **8:2 비율 및 층화 추출(Stratified Split) 방식으로 분할**하시오.

2. 수치형 변수에는 **중위수 대치(Median Imputation) 및 표준화(StandardScaler)**를 적용하고, 범주형 변수에는 **최빈값 대치(Most Frequent) 및 원-핫 인코딩(One-Hot Encoding)**을 수행하는 **전처리 파이프라인(`ColumnTransformer`)을 구축**하시오.
3. 학습용 데이터셋(`X_train`, `y_train`)에 대해서만 **SMOTE 를 적용하여 클래스 불균형을 해소**하고, 전후 클래스 빈도를 비교하시오.
4. 데이터 누수가 발생하지 않도록 전처리된 데이터셋을 생성하고 최종 차원(Shape)을 확인하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
```

```python
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from imblearn.over_sampling import SMOTE

# 1. 시뮬레이션 데이터셋 생성 (결측치 및 불균형 포함)
np.random.seed(42)
n_samples = 500

# 특성 생성
tenure = np.random.exponential(scale=20, size=n_samples) # 가입 기간 (수치형)
monthly_charges = np.random.normal(loc=70, scale=30, size=n_samples) # 월 요금 (수치형)
contract = np.random.choice(['Month-to-month', 'One year', 'Two year'], size=n_samples, p=[0.5, 0.3, 0.2]) # 계약 유형 (범주형)
payment_method = np.random.choice(['Electronic', 'Mailed check', 'Bank transfer'], size=n_samples) # 결제 수단 (범주형)

# 결측치 임의 주입 (5% 확률)
tenure[np.random.choice(n_samples, 25, replace=False)] = np.nan
contract[np.random.choice(n_samples, 20, replace=False)] = None

# 타겟 변수 생성 (약 15% 이탈 불균형)
churn_prob = 1 / (1 + np.exp(-(0.05 * monthly_charges - 0.08 * np.nan_to_num(tenure, nan=20) - 2.0)))
churn = (np.random.uniform(0, 1, size=n_samples) < churn_prob).astype(int)

df = pd.DataFrame({
    'tenure': tenure,
    'monthly_charges': monthly_charges,
    'contract': contract,
    'payment_method': payment_method,
    'churn': churn
})

# ========================================
# [소문항 1] 층화 분할 (Stratified Train/Test Split)
# ========================================
print("=== [1] Train / Test 데이터 분할 ===")
X = df.drop(columns=['churn'])
y = df['churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"X_train 크기: {X_train.shape}, X_test 크기: {X_test.shape}")
```

```python
print(f"학습셋 타겟 분포:\n{y_train.value_counts(normalize=True).round(3)}")

# ========================================
# [소문항 2] 전처리 파이프라인 (ColumnTransformer) 구축
# ========================================
print("\n=== [2] 전처리 파이프라인 구축 ===")
num_features = ['tenure', 'monthly_charges']
cat_features = ['contract', 'payment_method']

# 수치형 파이프라인: 중위수 결측치 대치 -> 표준화 스케일링
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# 범주형 파이프라인: 최빈값 결측치 대치 -> 원-핫 인코딩
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', sparse_output=False,
handle_unknown='ignore'))
])

# ColumnTransformer 결합
preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_pipeline, num_features),
        ('cat', cat_pipeline, cat_features)
    ]
)

# 학습 데이터 기준으로만 fit 및 변환 (데이터 누수 방지)
X_train_preprocessed = preprocessor.fit_transform(X_train)
X_test_preprocessed = preprocessor.transform(X_test)

# 생성된 컬럼명 추출
cat_encoded_cols =
preprocessor.named_transformers_['cat']['encoder'].get_feature_names_out(cat_
features)
all_feature_names = num_features + list(cat_encoded_cols)

print("전처리 완료된 특성 목록:", all_feature_names)
print(f"전처리 후 X_train 형태: {X_train_preprocessed.shape}")

# ========================================
# [소문항 3 & 4] SMOTE 불균형 처리 (Train 셋에만 적용)
# ========================================
print("\n=== [3 & 4] SMOTE 적용 및 최종 데이터 확인 ===")
print("SMOTE 적용 전 클래스 분포:")
```

```
print(pd.Series(y_train).value_counts())

smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled =
smote.fit_resample(X_train_preprocessed, y_train)

print("\nSMOTE 적용 후 클래스 분포:")
print(pd.Series(y_train_resampled).value_counts())
print(f"\n 최종 학습 데이터 형태: X={X_train_resampled.shape},
y={y_train_resampled.shape}")
print(f"최종 평가 데이터 형태: X={X_test_preprocessed.shape}, y={y_test.shape}")
```

ADP 실기 채점 포인트 요약

- **데이터 누수(Data Leakage) 엄격 채점**:
- `fit_transform` 은 오직 `X_train` 에만 적용해야 합니다.
- `X_test` 에는 반드시 `transform` 만 호출해야 데이터 누수 감점을 피할 수 있습니다.
- **SMOTE 적용 대상**:
- SMOTE 는 **오직 학습 데이터(`X_train`, `y_train`)에만 적용**해야 합니다.
- 테스트 데이터(`X_test`, `y_test`)에 SMOTE 를 적용하면 실제 평가 데이터의 분포를 왜곡하므로 0 점 처리될 수 있습니다.
- **`ColumnTransformer` 활용**:
- 수치형 변수와 범주형 변수를 판다스로 수동 분리하여 작업하는 것보다 `ColumnTransformer` 를 사용하는 것이 코드 재사용성 및 누수 방지 측면에서 표준 채점 기준에 부합합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 8. [3] 머신러닝 모델링, 튜닝 및 평가의 핵심인 '앙상블 분류 모델 학습, 교차검증 기반 하이퍼파라미터 튜닝(GridSearchCV), 평가지표 산출 및 ROC-AUC/임계값(Threshold) 최적화'입니다.

실기 시험 머신러닝(50 점) 영역의 후반부 핵심 문항으로, **(1) 베이스라인 모델 선정 → (2) GridSearchCV / StratifiedKFold 튜닝 → (3) 혼동행렬, F1-Score, ROC-AUC 산출 → (4) predict_proba 를 활용한 Decision Threshold 최적화 및 비즈니스 해석**을 정확히 구현해야 합니다.

실전 기출 유형 문제

[문제] 이전 단계에서 전처리된 고객 이탈 데이터를 바탕으로 고객 이탈(churn)을 예측하는 최적의 머신러닝 모델을 구축하고자 합니다. 1. RandomForestClassifier 를 기본 모델로 설정하고, 5-Fold 계층적 교차검증(StratifiedKFold) 기반의 **GridSearchCV 를 수행하여 최적 하이퍼파라미터를 탐색**하시오. (탐색 대상: n_estimators, max_depth, min_samples_split)

2. 최적 모형으로 평가 데이터셋(X_test)을 예측하고, **혼동행렬(Confusion Matrix), 정확도(Accuracy), 정밀도(Precision), 재현율(Recall), F1-Score (Macro), ROC-AUC 점수를 산출**하시오.
3. 평가 데이터의 예측 확률(predict_proba)을 산출한 뒤, **F1-Score 를 극대화하는 최적의 분류 임계값(Decision Threshold)을 탐색하고 기본 임계값(0.5) 적용 시와의 성능을 비교**하시오.
4. 학습된 모델의 **특성 중요도(Feature Importance) 상위 5 개 변수를 추출하고 비즈니스적 시사점을 서술**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, roc_auc_score, roc_curve
)

# 1. 이전 단계 전처리 데이터 연계 (시뮬레이션 데이터셋 구성)
```

```
np.random.seed(42)
n_train, n_test = 400, 100
n_features = 6
feature_names = ['tenure', 'monthly_charges', 'contract_One_year',
'contract_Two_year',
                 'payment_Electronic', 'payment_Mailed']

X_train = np.random.randn(n_train, n_features)
y_train = np.random.choice([0, 1], size=n_train, p=[0.5, 0.5]) # SMOTE 처리
완료 상태

X_test = np.random.randn(n_test, n_features)
y_test = np.random.choice([0, 1], size=n_test, p=[0.85, 0.15]) # 실제 테스트
분포 (불균형)

# ========================================
# [소문항 1] GridSearchCV 하이퍼파라미터 튜닝
# ========================================
print("=== [1] 하이퍼파라미터 튜닝 (GridSearchCV) ===")
rf = RandomForestClassifier(random_state=42)

param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring='f1_macro',
    cv=cv,
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
print(f"최적 파라미터: {grid_search.best_params_}")
print(f"최고 교차검증 F1-Macro 점수: {grid_search.best_score_:.4f}\n")

# ========================================
# [소문항 2] 기본 임계값(0.5) 평가 지표 산출
# ========================================
print("=== [2] 테스트 데이터 성능 평가 (Threshold = 0.5) ===")
y_pred_proba = best_model.predict_proba(X_test)[:, 1]
y_pred = (y_pred_proba >= 0.5).astype(int)
```

```python
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, average='macro')
roc_auc = roc_auc_score(y_test, y_pred_proba)

print(f"혼동행렬:\n{cm}")
print(f"정확도 (Accuracy): {acc:.4f}")
print(f"정밀도 (Precision): {prec:.4f}")
print(f"재현율 (Recall): {rec:.4f}")
print(f"F1-Score (Macro): {f1:.4f}")
print(f"ROC-AUC 점수: {roc_auc:.4f}\n")

# ========================================
# [소문항 3] 최적 분류 임계값(Decision Threshold) 탐색
# ========================================
print("=== [3] 최적 임계값 탐색 ===")
thresholds = np.arange(0.1, 0.9, 0.05)
best_thresh = 0.5
best_f1 = 0.0

for th in thresholds:
    y_pred_th = (y_pred_proba >= th).astype(int)
    current_f1 = f1_score(y_test, y_pred_th, average='macro')
    if current_f1 > best_f1:
        best_f1 = current_f1
        best_thresh = th

print(f"최적 임계값: {best_thresh:.2f}")
print(f"임계값 조정 후 최고 F1-Score (Macro): {best_f1:.4f}")
print(f"-> 기본(0.5) 대비 F1-Score 변화: {f1:.4f} -> {best_f1:.4f} ({(best_f1
- f1):+.4f})\n")

# ========================================
# [소문항 4] 특성 중요도 (Feature Importance) 추출
# ========================================
print("=== [4] 특성 중요도 상위 변수 추출 ===")
importances = best_model.feature_importances_
feat_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False).reset_index(drop=True)

print(feat_df)
print(f"\n[비즈니스 시사점]: 가장 영향력이 높은 변수는
'{feat_df.iloc[0]['Feature']}'(중요도 {feat_df.iloc[0]['Importance']:.4f})이며,
```

```
"
      f"이탈 방지를 위해 해당 특성을 중점적으로 관리해야 합니다.")
```

ADP 실기 채점 포인트 요약

- **`scoring` 지표 명시**: 불균형 데이터셋 문제일 경우 `GridSearchCV` 의 `scoring` 파라미터에 단순 `accuracy` 대신 `f1_macro` 또는 `roc_auc` 를 명시해야 실전에서 적합한 모델이 선택됩니다.
- **`predict_proba` 활용**: 단순 `predict` 결과로는 ROC-AUC 및 Decision Threshold 탐색을 수행할 수 없습니다. 반드시 클래스 1 의 확률값(`predict_proba(X)[:, 1]`)을 추출해 계산해야 합니다.
- **지표 해석 서술**: 산출된 수치만 나열하지 않고, 모델이 양성 클래스(이탈 고객)를 얼마나 잘 검출하고 있는지(Recall/Precision 관점)를 1~2 문장으로 덧붙여야 감점을 방지할 수 있습니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 9. [3] 머신러닝의 '연속형 타겟 예측을 위한 정규화 회귀(Ridge, Lasso, ElasticNet) 및 트리 기반 회귀 모델(RandomForest, XGBoost, LightGBM) 비교와 회귀 평가지표(RMSE, MAE, R^2, MAPE) 산출'입니다.

실기 시험 회귀 문제에서는 단순 선형 회귀에 그치지 않고, **다중공선성을 완화하는 정규화 선형 모델(Lasso/Ridge)과 비선형 트리 앙상블 모델의 성능을 교차검증 기반으로 비교 → 잔차 및 평가지표 해석**을 요구합니다.

실전 기출 유형 문제

[문제] 주택 거래 데이터를 활용하여 주택 매매 가격(`target_price`, 천 달러)을 예측하는 최적의 회귀 모델을 개발하고자 합니다. 1. 데이터를 학습용(`X_train`, `y_train`)과 평가용(`X_test`, `y_test`)으로 **8:2 분할**하시오.

2. 선형 규제 모델인 **`Ridge`, `Lasso`** 와 트리 기반 앙상블 모델인 **`RandomForestRegressor`, `GradientBoostingRegressor`** 총 4 개 모델을

정의하고, 5-Fold 교차검증을 통해 RMSE(Root Mean Squared Error) 기준 최고 성능 모델을 선정**하시오.

3. 선정된 최적 모델로 평가 데이터셋(X_test)에 대해 예측을 수행하고, **MAE, MSE, RMSE, MAPE, R^2(결정계수)를 산출**하시오.
4. Lasso 모델의 특성 선택 효과(계수가 0 이 된 변수 확인) 및 최적 모델의 **잔차(Residuals) 분포를 진단**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
mean_absolute_percentage_error

# 1. 시뮬레이션 회귀 데이터셋 생성
np.random.seed(42)
n_samples = 300
n_features = 8

X_raw = np.random.randn(n_samples, n_features)
feature_names = [f'feat_{i+1}' for i in range(n_features)]

# 타겟 변수 생성 (일부 특성만 강한 영향력 + 비선형성 및 노이즈 추가)
y_raw = (
    50 + 15 * X_raw[:, 0] - 10 * X_raw[:, 1] + 5 * (X_raw[:, 2] ** 2)
    + 0.1 * X_raw[:, 3] + np.random.normal(0, 3, size=n_samples)
)

df = pd.DataFrame(X_raw, columns=feature_names)
df['price'] = y_raw

# ========================================
# [소문항 1] Train / Test 데이터 분할
# ========================================
print("=== [1] Train / Test 분할 ===")
X = df.drop(columns=['price'])
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"X_train 크기: {X_train.shape}, X_test 크기: {X_test.shape}\n")
```

```
# ========================================
# [소문항 2] 4 개 회귀 모델 교차검증 비교
# ========================================
print("=== [2] 5-Fold 교차검증 기반 모델 비교 (평가기준: RMSE) ===")

models = {
    'Ridge': Ridge(alpha=1.0, random_state=42),
    'Lasso': Lasso(alpha=0.5, random_state=42),
    'RandomForest': RandomForestRegressor(n_estimators=100, max_depth=5,
random_state=42),
    'GradientBoosting': GradientBoostingRegressor(n_estimators=100,
max_depth=3, random_state=42)
}

cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_results = {}

for name, model in models.items():
    # scikit-learn 에서는 neg_mean_squared_error 를 반환하므로 부호 반전 및 sqrt
적용
    neg_mse_scores = cross_val_score(model, X_train, y_train,
scoring='neg_mean_squared_error', cv=cv)
    rmse_scores = np.sqrt(-neg_mse_scores)
    cv_results[name] = rmse_scores.mean()
    print(f"- {name:<18} 평균 CV-RMSE: {rmse_scores.mean():.4f} (±
{rmse_scores.std():.4f})")

best_model_name = min(cv_results, key=cv_results.get)
print(f"\n 최적 모델 선정: {best_model_name} (최저 RMSE 달성)\n")

# ========================================
# [소문항 3] 최적 모델 평가 지표 산출
# ========================================
print(f"=== [3] {best_model_name} 테스트 데이터 최종 평가 ===")
best_model = models[best_model_name]
best_model.fit(X_train, y_train)

y_pred = best_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mape = mean_absolute_percentage_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MAE  (평균 절대 오차)       : {mae:.4f}")
print(f"MSE  (평균 제곱 오차)       : {mse:.4f}")
print(f"RMSE (평균 제곱근 오차)     : {rmse:.4f}")
print(f"MAPE (평균 절대 백분율 오차): {mape:.4f} ({mape * 100:.2f}%)")
```

```
print(f"R²   (결정계수)              : {r2:.4f}")

# ========================================
# [소문항 4] Lasso 변수 선택 및 잔차 진단
# ========================================
print("\n=== [4] Lasso 변수 선택 결과 및 잔차 분석 ===")
lasso_model = models['Lasso'].fit(X_train, y_train)
lasso_coefs = pd.Series(lasso_model.coef_, index=feature_names)
zero_coefs = lasso_coefs[lasso_coefs == 0].index.tolist()

print("[Lasso 회귀계수 현황]")
print(lasso_coefs.round(4))
print(f"-> Lasso에 의해 제거(계수=0)된 변수: {zero_coefs}")

residuals = y_test - y_pred
print(f"\n[최적 모델 잔차 요약]: 평균={residuals.mean():.4f},
표준편차={residuals.std():.4f}")
print("-> 잔차의 평균이 0에 근접하고 대칭적인 분포를 보입니다.")
```

ADP 실기 채점 포인트 요약

- **cross_val_score의 RMSE 계산**: Scikit-Learn의 `scoring` 인자에는 `rmse`가 직접 없으므로 `neg_mean_squared_error`를 지정한 뒤 부호를 바꾸고 제곱근(`np.sqrt(-scores)`)을 취해야 합니다.
- **회귀 평가지표 세트 암기**: 시험 문제에서 요구하는 평가지표(MAE, MSE, RMSE, MAPE, R^2)의 함수명을 정확히 호출할 수 있어야 합니다.
- **MAPE(평균 절대 백분율 오차) 주의점**: 실제 타겟값에 0이 포함되어 있는 경우 나눗셈 에러가 발생할 수 있으므로, 데이터 탐색 단계에서 0 존재 여부를 확인해야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 10. [3] 비지도학습 연계의 핵심인 '주성분 분석(PCA)을 활용한 차원 축소와 K-Means 군집 분석(최적 클러스터 수 탐색 및 실루엣 계수 평가)'입니다.

실기 시험에서는 머신러닝 단독 문제 외에도, **(1) 다차원 변수에 대해 스케일링 → (2) 주성분 분석(PCA) 수행 및 설명분산비율/Scree Plot 확인 → (3) K-Means 군집화 및**

엘보우(Elbow) 기법/실루엣 계수(Silhouette Score) 산출 → (4) 군집별 특성 프로파일링 및 해석으로 이어지는 비지도학습 파이프라인이 정형 데이터 분석 문제로 자주 출제됩니다.

실전 기출 유형 문제

[문제] 고객 300 명의 다차원 행동 지표(구매 빈도, 구매 금액, 최근 방문일수, 웹 체류시간, 할인 쿠폰 사용률, 반품률 등 6 개 변수)를 바탕으로 고객 세분화(Segmentation)를 수행하고자 합니다. 1. 다차원 수치형 변수에 대해 **표준화(StandardScaler)를 적용하고, 주성분 분석(PCA)을 수행하여 각 주성분의 설명분산비율(Explained Variance Ratio)과 누적 기여율을 제시**하시오.

2. 누적 설명분산비율이 80% 이상이 되는 **최적 주성분 개수를 결정**하시오.
3. 주성분 축소 데이터를 바탕으로 **K-Means 군집 분석을 수행**하되, 군집 수 k 를 2 부터 6 까지 변화시키며 **엘보우 기법(Inertia)과 실루엣 계수(Silhouette Score)를 산출하여 최적의 군집 수 k 를 선정**하시오.
4. 선정된 최적 군집별로 원본 변수의 평균값을 산출하여 **각 군집의 고객 특성을 요약 및 비교 해석**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. 다차원 시뮬레이션 고객 데이터셋 생성
np.random.seed(42)
n_samples = 300

# 6 개 연속형 변수 생성 (각각 다른 스케일과 상관관계 부여)
recency = np.random.exponential(scale=30, size=n_samples)          # 최근 방문일 (일)
frequency = np.random.poisson(lam=12, size=n_samples)              # 구매 횟수 (회)
monetary = frequency * np.random.normal(loc=50, scale=10, size=n_samples) #
```

```
# 총 구매금액 (만원)
dwell_time = np.random.normal(loc=15, scale=5, size=n_samples)      # 평균 체류시간 (분)
coupon_usage = np.random.uniform(0, 1, size=n_samples)              # 쿠폰 사용률
return_rate = np.random.beta(a=2, b=10, size=n_samples)             # 반품률

df = pd.DataFrame({
    'recency': recency,
    'frequency': frequency,
    'monetary': monetary,
    'dwell_time': dwell_time,
    'coupon_usage': coupon_usage,
    'return_rate': return_rate
})

# ========================================
# [소문항 1 & 2] 표준화 및 PCA 주성분 분석
# ========================================
print("=== [1 & 2] 표준화 및 PCA 주성분 분석 ===")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 전체 주성분 적합
pca = PCA(random_state=42)
pca.fit(X_scaled)

exp_var = pca.explained_variance_ratio_
cum_exp_var = np.cumsum(exp_var)

pca_summary = pd.DataFrame({
    '주성분': [f'PC{i+1}' for i in range(len(exp_var))],
    '개별 설명분산비율': exp_var,
    '누적 설명분산비율': cum_exp_var
})
print(pca_summary.round(4))

# 누적 설명분산비율 80% 이상을 만족하는 최소 주성분 개수 결정
n_components_optimal = np.argmax(cum_exp_var >= 0.80) + 1
print(f"\n-> 누적 기여율 80% 이상을 만족하는 최적 주성분 개수:
{n_components_optimal}개 (누적 비율: {cum_exp_var[n_components_optimal-
1]*100:.2f}%)")

# 최적 주성분으로 데이터 변환
pca_optimal = PCA(n_components=n_components_optimal, random_state=42)
X_pca = pca_optimal.fit_transform(X_scaled)
```

```
# ========================================
# [소문항 3] K-Means 최적 군집 수 탐색 (Inertia & Silhouette Score)
# ========================================
print("\n=== [3] 최적 군집 수(k) 탐색 ===")
k_range = range(2, 7)
inertia_list = []
silhouette_list = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_pca)

    inertia_list.append(kmeans.inertia_)
    score = silhouette_score(X_pca, labels)
    silhouette_list.append(score)
    print(f"- k={k} | 관성(Inertia): {kmeans.inertia_:.2f} | 실루엣 계수:
{score:.4f}")

# 실루엣 계수가 가장 높은 k 선정
best_k = k_range[np.argmax(silhouette_list)]
best_silhouette = max(silhouette_list)
print(f"\n 최적 군집 수 선정: k={best_k} (최대 실루엣 계수 =
{best_silhouette:.4f})")

# ========================================
# [소문항 4] 최종 군집화 및 군집별 특성 프로파일링
# ========================================
print(f"\n=== [4] k={best_k} 최종 군집 프로파일링 ===")
final_kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df['cluster'] = final_kmeans.fit_predict(X_pca)

# 군집별 원본 변수 평균값 산출
cluster_profile = df.groupby('cluster').mean()
cluster_counts = df['cluster'].value_counts().rename('고객수')

profile_summary = pd.concat([cluster_counts, cluster_profile],
axis=1).sort_index()
print(profile_summary.round(2))

print("\n[군집 해석]:")
for c in range(best_k):
    sub = df[df['cluster'] == c]
    print(f"- 군집 {c} (N={len(sub)}): 평균 구매금액
{sub['monetary'].mean():.1f}만원, "
          f"구매빈도 {sub['frequency'].mean():.1f}회, 최근방문일
{sub['recency'].mean():.1f}일")
```

ADP 실기 채점 포인트 요약

- **PCA 전 표준화 필수**: 변수마다 단위(금액, 일수, 비율 등)가 상이하므로 PCA 를 수행하기 전에 반드시 `StandardScaler` 를 적용해야 분산 왜곡 감점을 피할 수 있습니다.
- **주성분 개수 결정 근거 서술**: 문제에서 특정 기여율(보통 70~80% 이상)이나 고유값(Eigenvalue ≥ 1) 기준을 명시하므로, 산출된 누적 설명분산비율 수치를 답안에 기재해야 합니다.
- **실루엣 계수 해석**:
- 실루엣 계수는 -1 에서 1 사이 값을 가지며, 1 에 가까울수록 군집 내 응집도가 높고 군집 간 분리가 잘 이루어졌음을 의미합니다.
- `KMeans` 실행 시 `random_state` 와 `n_init` 매개변수를 명시하여 재현성을 확보해야 합니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 11. [1] 통계 분석의 특수 회귀 영역인 '포아송 회귀분석 (Poisson Regression: 사건 발생 건수/가산 데이터 모델링 및 과대산포 검정)'입니다.

실기 시험에서 종속변수(y)가 음수가 될 수 없고 0 이상의 정수(예: 일일 방문자 수, 특정 결함 발생 건수, 사고 건수 등)로 주어질 때 일반 선형회귀 대신 **포아송 회귀분석**을 적합하고, **계수 지수 변환(e^β)을 통한 발생률비(Incidence Rate Ratio, IRR) 해석** 및 **과대산포(Overdispersion) 진단**을 요구하는 문제가 출제됩니다.

실전 기출 유형 문제

[문제] 한 온라인 쇼핑몰에서 고객들의 월간 구매 건수(`purchase_count`, 0 이상의 정수)를 종속변수로 하여, 고객의 웹사이트 체류시간(`dwell_time`, 분), 지난달 할인 쿠폰 발급 수(`coupon_count`, 개), 회원 가입 기간(`membership_years`, 년)의 영향을 분석하고자 합니다. 1. 세 독립변수를 활용하여 월간 구매 건수를 종속변수로 하는 **포아송 회귀모형(GLM Poisson)을 적합하고 결과 요약표를 제시**하시오.

2. 모형의 **이탈도(Deviance), 피어슨 카이제곱 통계량 및 자유도를 산출하여 과대산포(Overdispersion)가 존재하는지 진단**하시오.
3. 각 독립변수의 회귀계수를 지수 변환(e^β)하여 **발생률비(IRR: Incidence Rate Ratio)를 산출하고, 체류시간과 쿠폰 발급 수가 구매 건수에 미치는 영향을 해석**하시오.
4. 체류시간이 30 분, 할인 쿠폰 3 개, 가입 기간 2 년인 고객의 **월간 예상 구매 건수를 예측**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. 포아송 분포 기반 시뮬레이션 데이터셋 생성
np.random.seed(42)
n = 200

dwell_time = np.random.uniform(5, 60, size=n)        # 체류시간 (분)
coupon_count = np.random.poisson(lam=2, size=n)      # 발급 쿠폰 수 (개)
membership_years = np.random.uniform(0.5, 5.0, size=n) # 가입 기간 (년)

# 로그-선형 관계식 설정: log(lambda) = beta0 + beta1*dwell + beta2*coupon +
beta3*membership
log_mu = -0.5 + 0.03 * dwell_time + 0.25 * coupon_count + 0.15 *
membership_years
mu = np.exp(log_mu)

# 포아송 확률분포를 따르는 발생 건수 생성
purchase_count = np.random.poisson(lam=mu)

df = pd.DataFrame({
    'purchase_count': purchase_count,
    'dwell_time': dwell_time,
    'coupon_count': coupon_count,
    'membership_years': membership_years
})

# ========================================
# [소문항 1] 포아송 회귀모형 적합 (GLM)
# ========================================
print("=== [1] 포아송 회귀분석 적합 ===")
poisson_model = smf.glm(
```

```
    formula='purchase_count ~ dwell_time + coupon_count + membership_years',
    data=df,
    family=sm.families.Poisson()
).fit()

print(poisson_model.summary())

# ========================================
# [소문항 2] 과대산포 (Overdispersion) 진단
# ========================================
print("\n=== [2] 과대산포 (Overdispersion) 진단 ===")
deviance = poisson_model.deviance
df_resid = poisson_model.df_resid
pearson_chi2 = poisson_model.pearson_chi2
dispersion_ratio = pearson_chi2 / df_resid

print(f"이탈도 (Deviance): {deviance:.4f}")
print(f"잔차 자유도 (Residual df): {df_resid}")
print(f"피어슨 카이제곱 / 자유도 (Dispersion Ratio): {dispersion_ratio:.4f}")

if dispersion_ratio > 1.5:
    print("-> 분산비율이 1 보다 현저히 크므로 과대산포(Overdispersion)가
의심됩니다. (음이항 회귀 고려 필요)")
else:
    print("-> 분산비율이 약 1 에 가까우므로 포아송 분포 가정(평균=분산)이
타당합니다.")

# ========================================
# [소문항 3] 회귀계수 지수 변환 (IRR 산출) 및 해석
# ========================================
print("\n=== [3] 발생률비 (IRR: Incidence Rate Ratio) 산출 및 해석 ===")
params = poisson_model.params
irr = np.exp(params)
p_values = poisson_model.pvalues

irr_df = pd.DataFrame({
    'Coefficient': params,
    'IRR (exp(beta))': irr,
    'p-value': p_values
})
print(irr_df.round(4))

print("\n[IRR 해석]:")
for col in ['dwell_time', 'coupon_count', 'membership_years']:
    irr_val = irr[col]
    pct = (irr_val - 1) * 100
    if pct > 0:
        print(f"- {col}: 1 단위 증가 시 기대 구매 건수가 약 {pct:.2f}% 증가합니다.
```

```
(IRR = {irr_val:.4f})")
    else:
        print(f"- {col}: 1 단위 증가 시 기대 구매 건수가 약 {abs(pct):.2f}%
감소합니다. (IRR = {irr_val:.4f})")

# ========================================
# [소문항 4] 특정 고객의 기대 구매 건수 예측
# ========================================
print("\n=== [4] 특정 조건 고객의 구매 건수 예측 ===")
new_customer = pd.DataFrame({
    'dwell_time': [30.0],
    'coupon_count': [3],
    'membership_years': [2.0]
})

predicted_count = poisson_model.predict(new_customer)[0]
print(f"예측된 월간 구매 건수(기댓값): {predicted_count:.2f}회")
```

ADP 실기 채점 포인트 요약

- **GLM 패키지 사용**: 포아송 회귀는 `statsmodels.formula.api.glm` 에서 `family=sm.families.Poisson()`을 지정하여 호출해야 합니다.
- **발생률비(IRR) 지수 변환**: 로지스틱 회귀의 오즈비와 마찬가지로 포아송 회귀계수는 로그 단위이므로, 해석 시 반드시 `np.exp(params)`를 취해 **IRR = e^β** 값을 구하고 퍼센트 변화율로 서술해야 합니다.
- **과대산포 진단 기준**: 포아송 분포는 평균 = 분산을 가정하므로, 피어슨 카이제곱 통계량 / 잔차 자유도 비율이 1 을 크게 상회(보통 1.5 이상)하는지 여부를 서술하는 것이 핵심 채점 기준입니다.

확인해 보시고, 다음 항목으로 넘어가시려면 말씀해 주세요.

주제 12. [2] 데이터 전처리 및 [4] 파이썬 데이터 핸들링의 핵심인 '날짜/시간(Datetime) 파생변수 생성 및 그룹별(Groupby) 집계 피처 엔지니어링 파이프라인'입니다.

실기 시험 머신러닝 문제에서는 정제되지 않은 시점 로그나 거래 내역이 주어지는 경우가 많습니다. 모델에 바로 넣을 수 없는 원시 날짜 데이터를 파싱하여 **주말 여부, 분기, 경과**

일수 등을 추출하고, 고객/그룹별 통계량(평균, 합계, 비율)을 산출하여 원본 데이터에 병합(Merge)하는 과정이 필수적으로 출제됩니다.

실전 기출 유형 문제

[문제] 고객의 과거 주문 로그 데이터(orders.csv)와 고객 정보 데이터(customers.csv)가 주어졌습니다. 머신러닝 분류 모델에 입력할 수 있도록 특성 공학(Feature Engineering)을 수행하고자 합니다. 1. 주문 일자(order_date, 문자열)를 날짜형(Datetime)으로 변환하고, **주문 연도(year), 월(month), 요일(dayofweek), 주말 여부(is_weekend, 주말:1, 평일:0)** 파생변수를 생성하시오.

2. 기준일자('2026-01-01')로부터 각 주문 건까지의 **경과 일수(days_elapsed)를 계산**하시오.
3. 고객별(customer_id)로 **총 주문 횟수, 총 결제 금액, 평균 결제 금액, 주말 주문 비율, 최근 주문까지의 경과 일수(최솟값)**를 집계하는 요약 피처 테이블을 생성하시오.
4. 요약된 피처 테이블을 고객 기본 정보 테이블과 **customer_id 를 기준으로 Left Join 하고, 결측치가 발생한 고객의 결제 금액/횟수는 0 으로 대치**하시오.

완전 실행 가능한 파이썬 실전 코드

```
import numpy as np
import pandas as pd

# 1. 온라인/시뮬레이션 데이터셋 생성
np.random.seed(42)

# (1) 고객 기본 정보 테이블 (총 100 명)
customers_df = pd.DataFrame({
    'customer_id': [f'CUST_{i:03d}' for i in range(1, 101)],
    'age': np.random.randint(20, 60, size=100),
    'membership_grade': np.random.choice(['Bronze', 'Silver', 'Gold'],
size=100, p=[0.5, 0.3, 0.2])
})

# (2) 주문 로그 테이블 (총 500 건, 일부 고객은 주문 없음)
n_orders = 500
random_cust_ids = np.random.choice(customers_df['customer_id'][:80],
size=n_orders) # 80 명만 주문 이력 존재
```

```python
random_dates = pd.date_range(start='2025-01-01', end='2025-12-31',
periods=n_orders)
random_amounts = np.random.exponential(scale=50000, size=n_orders) + 10000

orders_df = pd.DataFrame({
    'order_id': [f'ORD_{i:04d}' for i in range(1, n_orders + 1)],
    'customer_id': random_cust_ids,
    'order_date': random_dates.strftime('%Y-%m-%d %H:%M:%S'), # 문자열 형태의
날짜
    'amount': random_amounts.round(-2)
})

# ========================================
# [소문항 1 & 2] 날짜 파싱 및 파생변수 생성
# ========================================
print("=== [1 & 2] 날짜형 변환 및 시점 파생변수 생성 ===")
# 문자열 -> Datetime 변환
orders_df['order_date'] = pd.to_datetime(orders_df['order_date'])

orders_df['year'] = orders_df['order_date'].dt.year
orders_df['month'] = orders_df['order_date'].dt.month
orders_df['dayofweek'] = orders_df['order_date'].dt.dayofweek # 0:월, ...,
5:토, 6:일
orders_df['is_weekend'] = orders_df['dayofweek'].apply(lambda x: 1 if x >= 5
else 0)

# 기준일자 대비 경과 일수 계산
base_date = pd.to_datetime('2026-01-01')
orders_df['days_elapsed'] = (base_date - orders_df['order_date']).dt.days

print(orders_df[['customer_id', 'order_date', 'year', 'month', 'is_weekend',
'days_elapsed']].head(4))

# ========================================
# [소문항 3] 고객별(Groupby) 집계 피처 엔지니어링
# ========================================
print("\n=== [3] 고객별 집계 피처 생성 ===")
cust_features = orders_df.groupby('customer_id').agg(
    order_count=('order_id', 'count'),
    total_amount=('amount', 'sum'),
    mean_amount=('amount', 'mean'),
    weekend_order_ratio=('is_weekend', 'mean'),
    recency_days=('days_elapsed', 'min') # 가장 최근 주문의 경과 일수
).reset_index()

print(cust_features.head(4).round(2))

# ========================================
# [소문항 4] 테이블 병합 (Left Join) 및 결측치 처리
```

```python
# ========================================
print("\n=== [4] 고객 정보와 병합 및 결측치 대치 ===")
final_df = pd.merge(customers_df, cust_features, on='customer_id', how='left')

# 주문 이력이 없는 고객 결측치 대치
final_df['order_count'] = final_df['order_count'].fillna(0).astype(int)
final_df['total_amount'] = final_df['total_amount'].fillna(0)
final_df['mean_amount'] = final_df['mean_amount'].fillna(0)
final_df['weekend_order_ratio'] = final_df['weekend_order_ratio'].fillna(0)
final_df['recency_days'] =
final_df['recency_days'].fillna(final_df['recency_days'].max()) # 미구매
고객은 최대 일수로 대치

print(f"최종 병합 데이터셋 크기: {final_df.shape}")
print(final_df.head(6).round(2))
```

ADP 실기 채점 포인트 요약

- **`pd.to_datetime()` 및 `.dt` 접근자 숙지**:
- 문자열 날짜를 판다스 `datetime64` 포맷으로 변환한 뒤 `.dt.year`, `.dt.month`, `.dt.dayofweek`, `.dt.days` 등을 정확히 활용할 수 있어야 합니다.
- **`groupby().agg()` 다중 집계 문법**:
- `agg(새컬럼명=('기존컬럼', '집계함수'))` 형태의 튜플 문법을 사용하면 한 번에 명확한 컬럼명을 가진 피처 요약본을 만들 수 있어 시간을 크게 단축할 수 있습니다.
- **Merge 후 결측치 처리 전략**:
- `how='left'` 조인 시 주문 이력이 없는 고객은 `NaN` 이 발생하므로, 금액/횟수는 `0`, 경과일수(Recency)는 최댓값 등으로 적절히 대치하는 서술을 함께 작성해야 감점이 없습니다.

12 개 주제를 통해 통계 검정 파이프라인(t-test/ANOVA/카이제곱/회귀/시계열/포아송)부터 머신러닝 파이프라인(전처리/누수방지/불균형/분류·회귀 튜닝/PCA-군집/특성공학)까지 실기 시험의 필수 전 영역을 다루었습니다. 추가로 보강하거나 다른 예제로 다뤄보고 싶은 영역이 있으신가요?

ADP 필기 제 4·5 과목 핵심 이론 정리

제외되었던 항목들은 ADP 필기 제 4 과목(데이터 분석) 및 제 5 과목(데이터 시각화 및 안내)에서 개념, 특징 비교, 계산 공식 위주로 반복 출제되는 핵심 이론들입니다. 각 주제별 핵심 개념, 필기 빈출 포인트, 예상 문제 및 보기별 상세 해설로 정리했습니다.

---

1. 심층 인공신경망 및 딥러닝 이론

- **주요 모델 및 특징**

- **RBM (Restricted Boltzmann Machine)**: 가시층(Visible layer)과 은닉층(Hidden layer) 간의 층간 연결만 존재하고, 동일 층 내 노드 간 연결은 제한된 확률적 생성 모형입니다.

- **DBN (Deep Belief Network)**: RBM 을 여러 층 쌓아 올린 구조로, 비지도 사전학습(Pre-training) 후 역전파(Backpropagation)로 미세조정(Fine-tuning)을 수행합니다.

- **AutoEncoder**: 입력을 압축(인코더)한 후 원래 입력과 같아지도록 복원(디코더)하는 비지도 신경망으로, 차원 축소 및 특성 추출, 이상치 탐지에 활용됩니다.

- **VAE (Variational AutoEncoder)**: 잠재 공간(Latent Space)을 특정 확률분포(주로 표준정규분포)로 가정하여 새로운 데이터를 생성하는 확률적 생성 모델입니다.

- **GAN (Generative Adversarial Network)**: 생성자(Generator)와 판별자(Discriminator)가 적대적(Zero-sum)으로 경쟁하며 실제와 유사한 데이터를 생성하는 모형입니다.

- **ResNet**: Residual Connection(Skip Connection)을 도입하여 $F(x) + x$ 구조를 통해 기울기 소실(Vanishing Gradient) 문제를 해결하고 망을 깊게 쌓을 수 있게 한 모델입니다.

- **DQN (Deep Q-Network)**: 강화학습의 Q-Learning 에 심층 신경망(CNN 등)을 결합하여 고차원 상태 공간에서도 최적 정책을 학습하도록 한 알고리즘입니다.

- **유전 알고리즘 (Genetic Algorithm) & 마르코프 결정과정 (MDP)**
- **유전 알고리즘 3 대 연산자**: 선택(Selection), 교차(Crossover), 변이(Mutation).
- **MDP 5 대 구성요소**: 상태(State, S), 행동(Action, A), 전이확률(P), 보상함수(R), 할인율(Discount factor, γ).

---

2. 사회연결망분석 (SNA: Social Network Analysis)

- **네트워크 구조 파악 지표**
- **연결정도 중심성 (Degree Centrality)**: 한 노드가 직접적으로 연결된 다른 노드의 수.
- **근접 중심성 (Closeness Centrality)**: 한 노드에서 다른 모든 노드까지의 최단 거리 합의 역수로, 정보 전달의 신속성을 측정.
- **매개 중심성 (Betweenness Centrality)**: 네트워크 내 서로 다른 노드 간의 최단 경로 위에 위치하는 빈도로, 중개자/게이트키퍼 역할 측정.
- **아이겐벡터 중심성 (Eigenvector Centrality)**: 연결된 다른 노드의 중요도까지 가중치로 반영한 중심성.
- **구조적 틈새 (Structural Holes)**: 서로 직접 연결되지 않은 집단 사이를 연결하여 정보 독점이나 통제 이점을 얻을 수 있는 빈 공간.

---

3. 텍스트마이닝 및 자연어 처리 (NLP) 심화

- **주요 기법**
- **형태소 분석 및 정규화**: 어간 추출(Stemming: 규칙 기반 어미 제거), 표제어 추출(Lemmatization: 품사를 고려한 단어의 사전적 기본형 도출).
- **N-Gram**: 연속된 n 개의 단어나 글자를 하나의 토큰 단위로 묶어 문맥 정보를 보존하는 기법.
- **TF-IDF**: TF × log(N/DF). 특정 문서 내 빈도는 높으나 전체 문서군에서는 흔하지 않은 핵심어를 추출하는 가중치.
- **Word2Vec**: 단어를 밀집 벡터(Dense Vector)로 임베딩하는 방식으로, 주변 단어로 중심 단어를 예측하는 CBOW 와 중심 단어로 주변 단어를 예측하는 Skip-gram 으로 구분.

- **LDA (Latent Dirichlet Allocation)**: 문서가 여러 토픽의 혼합으로 구성되어 있고, 각 토픽은 단어들의 확률분포로 구성되어 있다고 가정하는 확률적 토픽 모델링 기법.
- **Perplexity (혼주도/당혹도)**: 언어 모델의 성능 평가 지표로, 값이 낮을수록 모델의 예측 성능이 우수함을 의미.

---

4. 다변량 시각화 기법 (제 5 과목 출제)

- **체르노프 페이스 (Chernoff Face)**: 다차원 데이터의 각 변수를 사람 얼굴의 특성(눈 크기, 입 꼬리 각도, 코 길이 등)에 매핑하여 시각화하는 기법. 인간의 얼굴 인식 능력에 의존하므로 변수 순서나 주관적 해석에 따라 왜곡이 발생할 수 있음.
- **스타 차트 (Star Chart / Radar Chart)**: 원 중심에서 방사형으로 뻗어 나가는 축에 변수 값을 표시하여 여러 변수의 상대적 수준을 한눈에 비교하는 다변량 그래프.

ADP 필기 실전 대비 모의 문제 및 상세 해설

[문제 1] 심층 신경망(Deep Learning) 구조 및 알고리즘에 대한 설명으로 가장 적절하지 않은 것은?

① AutoEncoder 는 입력 데이터와 동일한 출력을 생성하도록 학습하며, 잠재 공간(Latent Space)을 통해 비선형 차원 축소 및 이상치 탐지에 활용할 수 있다.

② ResNet 은 Residual Learning(Skip Connection) 구조를 적용하여 네트워크가 깊어질 때 발생하는 기울기 소실(Vanishing Gradient) 문제를 효과적으로 완화한다.

③ RBM(Restricted Boltzmann Machine)은 가시층과 은닉층 간의 연결은 존재하지만, 동일 층 내 노드 간에는 상호 연결이 존재하지 않는 무방향 이분 그래프 구조를 가진다.

④ Word2Vec 의 Skip-gram 방식은 주변 단어들의 맥락(Context)을 바탕으로 중심 단어(Target)를 예측하도록 학습하며, 일반적으로 CBOW 방식보다 연산 속도가 빠르다.

- **정답**: ④
- **보기별 상세 해설**:
- ① (정답 아님) AutoEncoder 는 인코더와 디코더를 통해 입력을 압축 및 복원하며, 차원 축소와 노이즈 제거, 이상치 탐지 등에 널리 쓰입니다.

- ② (정답 아님) ResNet 은 H(x) = F(x) + x 형태의 잔차 학습을 통해 100 개 이상의 깊은 층에서도 기울기 소실을 방지합니다.
- ③ (정답 아님) RBM 은 동일 층 내 연결이 제한(Restricted)되어 있는 볼츠만 머신 구조입니다.
- ④ **(정답/오답 설명)** Skip-gram 은 **중심 단어를 통해 주변 단어를 예측**하는 방식이며, 주변 단어로 중심 단어를 맞추는 것은 CBOW 입니다. 또한 Skip-gram 은 희귀 단어 학습에 유리하나 CBOW 에 비해 연산량이 더 많아 속도가 느립니다.

[문제 2] 사회연결망분석(SNA)의 중심성(Centrality) 지표에 대한 설명으로 옳지 않은 것은?

① 연결정도 중심성(Degree Centrality)은 한 노드가 직접적으로 연결된 이웃 노드의 총 개수를 기반으로 측정된다.

② 매개 중심성(Betweenness Centrality)은 네트워크 내 모든 노드 쌍 간의 최단 경로 중에서 해당 노드를 경유하는 비율을 의미하며, 정보 전달의 중개자 역할을 파악하는 데 적합하다.

③ 근접 중심성(Closeness Centrality)은 한 노드에서 다른 모든 노드까지의 최단 거리 합이 클수록(거리가 멀수록) 높은 값을 가진다.

④ 아이겐벡터 중심성(Eigenvector Centrality)은 단순히 연결된 노드의 수뿐만 아니라, 자신과 연결된 다른 노드들의 중요도(영향력)까지 반영하여 산출한다.

- **정답**: ③
- **보기별 상세 해설**:
- ① (정답 아님) 연결정도 중심성은 한 노드와 직접적인 링크를 맺고 있는 노드 수에 비례합니다.
- ② (정답 아님) 매개 중심성은 최단 경로 상에서 얼마나 브릿지 역할을 하는지를 평가하는 지표입니다.
- ③ **(정답/오답 설명)** 근접 중심성은 다른 모든 노드까지의 최단 거리의 합의 역수(Inverse)로 계산됩니다. 즉, 다른 노드들과의 최단 거리 합이 **작을수록(전체 네트워크의 중심에 가까워 빠르게 도달할 수 있을수록) 근접 중심성이 높아집니다.**
- ④ (정답 아님) 아이겐벡터 중심성은 구글의 PageRank 와 유사하게 '중요한 노드와 많이 연결되어 있을수록 더 큰 영향력을 갖는다'는 원리를 반영합니다.